

The Turing library

Douglas Wilhelm Harder

A Turing machine consists of:

1. a finite alphabet,
2. a linear tape consisting of an infinite number of cells, where each cell contains either a symbol from the alphabet or a BLANK,
3. a finite collection of states, including an initial state and at least one designated halting state,
4. a head positioned over a single cell of the tape that can read the symbol or BLANK in that cell, write a symbol or BLANK to that cell, and move one cell to the left or one cell to the right, and
5. a program consisting of a collection of transitions that map a pair consisting of the current state and the symbol (or BLANK) currently under the head to a new state, a symbol (or BLANK) to be written, and a command directing the head to move left, move right, or remain over the current cell.

Given an initial tape configuration containing only finitely many non-BLANK cells, an initial head position, and an initial state, the machine repeatedly applies the corresponding transition. This process continues until the machine enters a halting state or executes forever.

1 class Alphabet

The class `Alphabet<Symbol_type>` is a templated class representing the finite set of symbols of the given type that may appear on the tape. The BLANK symbol is not a member of this alphabet; it is treated separately.

The constructors include:

1. Create an alphabet of type `int` consisting of the four symbols {1, 2, 5, 9}:

```
Alphabet A1{ 1, 2, 5, 9 };
```

If a duplicate symbol is found, an exception will be thrown.

2. Create an alphabet of type `unsigned int` consisting of the symbols {10, 11, 12, 13}:

```
Alphabet A2( 10U, 13U );
```

3. Create an alphabet of type `char` consisting of the characters {'a', 'b', ..., 'm'} where the endpoints are included:

```
Alphabet A3( 'a', 'm' );
```

4. For a character alphabet, create an alphabet of type `char` consisting of the four characters in the string: {'a', 'b', 'c', 'd'}:

```
Alphabet A4( "abcd" );
```

5. For an integer type alphabet, create an alphabet from the same integer type consisting of the two symbols {0, 1, 2, 3}:

```
Alphabet A5( 4 );
```

This notation is used so that `Alphabet A5( 2 );` would represent a binary alphabet.

6. You can always explicitly state the type using a template parameter:

```
Alphabet<unsigned int> A6{ 0, 2, 4, 6, 8 };    // {0, 2, 4, 6, 8 }
Alphabet<unsigned int> A7( 0, 3 );             // {0, 1, 2, 3}
Alphabet<unsigned int> A8( 2 );                // {0, 1}
```

The underscore '_' may not be used as a symbol in a character alphabet, as it is used to display BLANK cells.

The member function `size()` returns the number of symbols in the alphabet not counting the BLANK. All other member functions are used internally and are not needed for ordinary use.

The most common alphabets you are likely to use are:

```
Alphabet Binary_char{ "01" };           // '0', '1'
Alphabet Decimal_char{ "0123456789" };   // '0', '1', '2', ..., '9'
Alphabet Hexadecimal_char{ "0123456789abcdef" }; // '0', '1', '2', ..., '9', 'a', ..., 'f'
Alphabet Binary( 2 );                     // 0, 1
Alphabet Decimal( 10 );                   // 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
// These allow negative numbers to be stored on the tape
Alphabet Binary_signed{ "-01" };
Alphabet Decimal_signed{ "-0123456789" };
Alphabet Hexadecimal_char{ "-0123456789abcdef" };
```

In the examples throughout this manual, character alphabets are used to make it easy to distinguish between states and tape symbols.

2 The TM namespace

A number of special identifiers are used throughout the Turing library. These are encapsulated within the TM namespace.

In production software, it is generally advisable to avoid declarations such as

```
using namespace std;    // Standard library
using namespace TM;     // Turing library
```

because they import all identifiers from the namespace into the current scope. Consequently, future additions to the namespace may collide with locally defined variables, functions, classes, and other identifiers. For example, when `std::gcd` was added to the C++ standard library in C++17, it is probable that smaller projects or programs that had defined their own `gcd` functions experienced name collisions after upgrading to a newer compiler.

However, this library is intended primarily for experimentation, instruction, and undergraduate projects. Consequently, it is often more convenient to write

```
using namespace TM;
```

so that Turing programs more closely resemble their mathematical descriptions:

Category	Explicit namespace qualification	using namespace TM;
Directions	TM::LEFT TM::STAY TM::RIGHT	LEFT STAY RIGHT
The BLANK	TM::BLANK	BLANK
Wildcards and shortcuts	TM::ALL TM::SAME	ALL SAME
Predefined halting states	TM::HALT TM::TRUE TM::FALSE TM::FAIL	HALT TRUE FALSE FAIL

All examples below assume that the source code includes such a `using` statement.

3 class Tape

The class `Tape<Symbol_type>` represents the tape of a Turing machine. Each tape is associated with an `Alphabet<Symbol_type>` and stores only those cells that are either initialized or that are accessed. All other cells are assumed to contain BLANK. Each tape is associated with a prescribed cell that is to be placed under the head when a Turing program is run. This is to allow the tape to be passed back and forth between various Turing programs.

The tape is conceptually infinite in both directions. It is implemented using a hash table that stores only those cells that have been written. Any cell that has not been written is assumed to contain BLANK, allowing the tape to expand arbitrarily far in either direction while using memory proportional to the number of non-blank cells.

3.1 Streaming

A tape may be streamed to, for example, `std::cout` where you will have two tilde (`~~`) on either side representing that the tape goes infinitely on in both directions, and only those cells prescribed in the constructor or those that have been under the head will be printed. The BLANK will be printed by default with the underscore character `_`, which is why the underscore is not allowed as a character within character type alphabets.

3.2 Tape constructors

The constructor takes an alphabet as the first argument. If there is a second argument, it represents the initial symbols on the tape. By default, the head is placed under the first symbol, but the initial head location relative to the first symbol can be specified as a third argument. Thus, for example, we have:

1. Create a tape from the alphabet `{0, 1}` with eight bits representing the binary number for 42 with the head under the last bit, so this

```
Alphabet A5{ 0, 1 };
Tape tape5{ A5, {0, 0, 1, 0, 1, 0, 1, 0}, 7 };
std::cout << tape5 << std::endl;
```

produces the output `~~ 0 0 1 0 1 0 1 [0] ~~`.

2. If you want to add explicit blanks, you must specify the symbol type, so this

```
Alphabet A6{ 0, 1 };
Tape<int> tape6{ A6, {0, 0, 1, 0, BLANK, 1, 0, 1, 0}, 8 };
std::cout << tape6 << std::endl;
```

produces the output `~~ 0 0 1 0 _ 1 0 1 [0] ~~`. The explicit template argument is required because the compiler cannot always deduce the symbol type when both symbols and `BLANK` appear in the same initializer list.

3. Create a tape from the alphabet `{'a', 'b'}` with four symbols representing the greatest band on Earth, so this

```
Alphabet A7( "ab" );
Tape tape7{ A7, "abba", -1 };
std::cout << tape7 << std::endl;
```

produces the output `~~ [_] a b b a ~~` with the head above the blank before the first letter.

4. If you want one `BLANK` between clusters of characters, you can use an expression sequence of strings, so this

```
Alphabet A8( "ab" );
Tape tape8{ A8, {"ab", "ba"}, 2 };
std::cout << tape8 << std::endl;
```

produces the output `~~ a b [_] b a ~~` with the middle blank under the head.

4 Turing_program

A Turing program may be defined as an instance of the `Turing_program<Symbol_type>` class that will manipulate a tape of with symbols of the prescribed type. The constructor takes two arguments: an alphabet and a collection of halting states. States are represented by unsigned integers, and execution always begins in state 0.

4.1 Pre-defined halting states

In addition to user-defined unsigned integer halting states, four predefined halting states are provided for convenience:

1. `HALT`, indicating that execution has terminated.
2. `TRUE`, indicating that execution has terminated and that some proposition or condition has been determined to be true.
3. `FALSE`, indicating that execution has terminated and that some proposition or condition has been determined to be false.
4. `FAIL`, indicating that execution has terminated without determining whether a proposition or condition is true or false.

If no halting states are prescribed, then it is assumed `HALT` is the only halting state. If at least one halting state is prescribed, then those are the only halting states for the program. The latter three predefined states are provided to allow a uniform approach for Turing programs that intend to end in a Boolean-valued state that may subsequently be used to indicate a property of the tape.

4.2 Two interfaces

This library contains two approaches to define a Turing program. To illustrate both approaches, consider the transition that, when in State 0 reading the symbol '0', moves to State 1, writes the symbol '1', and moves the head one cell to the LEFT.

1. **Member function interface:** Given a Turing program P, the transition is defined by

`P( 0, '0' ) = { 1, '1', LEFT };`

2. **String interface:** All transitions are specified as strings and passed to the constructor. The corresponding transition is written as

`"0 0 -> 1 1 L"`

The next two sections describe these interfaces in detail. The examples throughout the remainder of this manual use the string interface because it more closely resembles the notation commonly found in textbooks.

4.3 The member function interface

We begin with the member function interface because it exposes the structure of a Turing program most directly. Although the string interface described in the next section more closely resembles the notation commonly found in textbooks, both interfaces are functionally equivalent and may be used interchangeably.

In this section, we describe how to declare a Turing program, define transitions, use shortcuts and wildcards for matching states and symbols, and finally construct larger programs by having one Turing program call another.

4.3.1 Examples using the member function interface

The following declares a Turing program over the alphabet of unsigned integers {0,1,2,3}. Since no halting states are specified, the only halting state is the predefined state HALT.

```
Alphabet A1( 4U );
Turing_program PROGRAM_NAME{ A1 };
```

The following declares a Turing program whose halting states are TRUE and FALSE.

```
Alphabet Binary{ "01" };
Turing_program IS_ONE_BIT{ Binary, {TRUE, FALSE} };
```

The optional identifier is required only if the program is to be called by another Turing program using the string interface. It is ignored when using the member function interface alone.

Once a program is declared, transitions can be defined as described in the next section.

4.3.2 Defining transitions

Once a Turing program has been declared, its transitions are defined using the assignment operator. Each assignment corresponds to a single transition in the transition table. The general form is

```
PROGRAM_NAME( CURRENT_STATE, CURRENT_SYMBOL )  
= { NEW_STATE, NEW_SYMBOL, DIRECTION };
```

Here, `CURRENT_STATE` and `CURRENT_SYMBOL` identify the current configuration of the machine. The machine then writes `NEW_SYMBOL`, moves the head in the specified `DIRECTION`, and continues execution in `NEW_STATE`.

The values of `CURRENT_SYMBOL` and `NEW_SYMBOL` must be either symbols from the alphabet or the special symbol `BLANK`. The direction must be one of `LEFT`, `RIGHT`, or `STAY`.

The current state may be any non-negative integer that is not one of the program's declared halting states. The new state may likewise be any non-negative integer or one of the predefined halting states `HALT`, `TRUE`, `FALSE`, or `FAIL`, provided that the corresponding predefined state was declared as a halting state when the program was constructed.

As an example, consider the following program, which determines whether the head is positioned over the only bit (either `'0'` or `'1'`) on the tape. The machine returns `TRUE` if the current symbol is an isolated bit surrounded by `BLANK` cells, and `FALSE` otherwise.

```
Alphabet Binary{ "01" };  
Turing_program IS_ONE_BIT{ Binary, { TRUE, FALSE } };  
  
// State 0  
// If the current cell is blank, we are not over a bit.  
// Otherwise, move right to examine the next cell.  
IS_ONE_BIT( 0, BLANK ) = { FALSE, BLANK, STAY };  
IS_ONE_BIT( 0, '0' ) = { 1, '0', RIGHT };  
IS_ONE_BIT( 0, '1' ) = { 1, '1', RIGHT };  
  
// State 1  
// If the cell to the right is blank, move back to the  
// original bit. Otherwise, another bit is present.  
IS_ONE_BIT( 1, BLANK ) = { 2, BLANK, LEFT };  
IS_ONE_BIT( 1, '0' ) = { FALSE, '0', LEFT };  
IS_ONE_BIT( 1, '1' ) = { FALSE, '1', LEFT };  
  
// State 2  
// We already know the current symbol is a bit, so  
// continue left to inspect the neighbouring cell.  
IS_ONE_BIT( 2, '0' ) = { 3, '0', LEFT };  
IS_ONE_BIT( 2, '1' ) = { 3, '1', LEFT };
```

```

// State 3
// If the cell to the left is blank, the original bit
// was isolated. Otherwise, another bit is present.
// In either case, return the head to its original position.
IS_ONE_BIT( 3, BLANK ) = { TRUE, BLANK, RIGHT };
IS_ONE_BIT( 3, '0' ) = { FALSE, '0', RIGHT };
IS_ONE_BIT( 3, '1' ) = { FALSE, '1', RIGHT };

```

Although this example defines each transition individually, the next subsection introduces shortcuts, sets, and wildcards that allow many similar transitions to be written much more concisely.

4.3.3 Matching multiple states and symbols

Many Turing machine transitions differ only in the symbol currently under the head or the current state. Rather than defining each such transition individually, the member function interface allows collections of states and symbols, together with several wildcards, to define multiple transitions simultaneously.

For a Turing program P over an alphabet A, the following forms may be used to specify the symbols to be matched:

```

P( S0, '0' )           // One symbol
P( S0, {'0', '1'} )    // A subset of symbols
P( S0, "01" )          // For character alphabets only
P( S0, A )              // Every symbol in the alphabet
P( S0, ALL )            // Every symbol, including BLANK

```

Similarly, multiple states may be specified by providing a brace-enclosed list:

```

P( {S0, S1}, '0' )
P( {S0, S1}, {'0', '1'} )

```

The symbol written by a transition may either be a specific symbol or the special symbol SAME, which leaves the current symbol unchanged:

```

P( S0, * ) = { S1, a1, MOVE };
P( S0, * ) = { S1, SAME, MOVE };

```

In the first case, every matching symbol is replaced by the symbol a1. In the second, the symbol under the head is preserved.

For example, in the program introduced previously, the following transition

```
IS_ONE_BIT( 0, {'0', '1'} ) = { 1, SAME, RIGHT };
```

is equivalent to the two transitions

```
IS_ONE_BIT( 0, '0' ) = { 1, '0', RIGHT };
IS_ONE_BIT( 0, '1' ) = { 1, '1', RIGHT };
```

Since both transitions perform exactly the same action apart from the symbol being read, the shorter form is both clearer and less repetitive.

The following program removes all contiguous bits beginning at the current cell and extending to the right. Each bit is replaced by a BLANK. When a BLANK is encountered, the machine halts.

```
Alphabet A{ "01" };
Turing_program CLEAN_TO_RIGHT{ A };

CLEAN_TO_RIGHT( 0, BLANK )    = { HALT, BLANK, STAY };
CLEAN_TO_RIGHT( 0, {'0','1'} ) = { 0, BLANK, RIGHT };
```

If the same transition should occur for every symbol in the alphabet, the alphabet itself may be used:

```
Alphabet A{ "01" };
Turing_program CLEAN_TO_RIGHT{ A };

CLEAN_TO_RIGHT( 0, BLANK ) = { HALT, BLANK, STAY };
CLEAN_TO_RIGHT( 0, A )    = { 0, BLANK, RIGHT };
```

Finally, if a transition should occur regardless of whether the current cell contains a symbol from the alphabet or BLANK, use the wildcard ALL:

```
Alphabet A{ "01" };
Turing_program MOVE_ONE_TO_RIGHT{ A };

MOVE_ONE_TO_RIGHT( 0, ALL ) = { 1, SAME, RIGHT };
MOVE_ONE_TO_RIGHT( 1, ALL ) = { HALT, SAME, STAY };
```

4.3.4 Programs calling other programs

One of the advantages of the member function interface is that larger Turing programs may be constructed from smaller, reusable components. A Turing program may invoke another Turing program, passing it the current tape. The called program begins execution at its initial state and operates directly on the tape. When it reaches one of its halting states, control returns to the calling program, with the tape and head position exactly as they were left by the called program.

As an example, consider the following program, which moves the head to the right-most non-BLANK cell.

```
// If the head is currently over a bit, continue moving
// right until the next cell is BLANK and then return TRUE.
//
// If the head is currently over BLANK, return FALSE
// without moving the head.

Alphabet Binary{ "01" };
Turing_program MOVE_TO_LSB{ Binary, { TRUE, FALSE } };

MOVE_TO_LSB( 0, BLANK ) = { FALSE, BLANK, STAY };
MOVE_TO_LSB( 0, Binary ) = { 1, SAME, RIGHT };

MOVE_TO_LSB( 1, BLANK ) = { TRUE, BLANK, LEFT };
MOVE_TO_LSB( 1, Binary ) = { 1, SAME, RIGHT };
```

If this program is only ever called when the head is known to be positioned over a bit, the initial test is unnecessary, and the program may be simplified to

```
MOVE_TO_LSB( 0, BLANK ) = { HALT, BLANK, LEFT };
MOVE_TO_LSB( 0, Binary ) = { 0, SAME, RIGHT };
```

When one program calls another, the halting states of the called program must be interpreted in the context of the calling program. Consequently, each halting state of the called program must either be accepted directly by the calling program or mapped to an appropriate state.

Specifically,

1. Any user-defined halting state of the called program must be mapped to a state of the calling program.
2. A predefined halting state (HALT, TRUE, FALSE, or FAIL) that is not a halting state of the calling program must also be mapped.
3. A predefined halting state shared by both programs may either be left unchanged, causing the calling program to halt in that state, or be mapped to another state so that execution continues.
4. If a particular halting state should never occur, it may be mapped to the special state **ERROR**. Should the called program terminate in this state, execution terminates immediately, indicating that an assumption made by the calling program has been violated.

For example, suppose the current program knows that the head is positioned over a bit. In that case, **MOVE_TO_LSB** should always terminate in state **TRUE** and should never return

FALSE. The following call continues execution in State 4 if the call succeeds and terminates with ERROR otherwise:

```
PROGRAM_NAME( 3, Binary ) =  
Call( MOVE_TO_LSB, {{ TRUE, 4 }}, { FALSE, ERROR }} );
```

Program calls provide a convenient mechanism for constructing larger Turing programs from smaller ones without changing the underlying computational model. Conceptually, calling a Turing program is equivalent to replacing the call with the transitions of the called program after consistently renaming its internal states and mapping its halting states to those of the calling program. The library performs this bookkeeping automatically, allowing programs to be written in a modular and reusable fashion while remaining faithful to the traditional definition of a Turing machine.

4.3.5 Summary

The member function interface closely mirrors the mathematical definition of a Turing machine while remaining entirely within the syntax of C++. Each transition is expressed as a single assignment, making the correspondence between a transition table and its implementation immediate. At the same time, the interface provides several conveniences, including sets of states and symbols, wildcards, and the ability to leave symbols unchanged using SAME, substantially reducing repetitive definitions without changing the underlying computation.

The ability for one Turing program to call another provides a natural mechanism for constructing larger programs from smaller, reusable components. Conceptually, such a call does not extend the computational model. It is equivalent to replacing the call with the transitions of the called program after consistently renaming its internal states and mapping its halting states to appropriate states in the calling program. Thus, program calls should be viewed as a modular programming abstraction rather than as an additional computational primitive.

Taken together, these features provide a concise and expressive interface for defining Turing machines while remaining faithful to their traditional mathematical formulation. Programs written using this interface closely resemble transition tables, yet scale naturally to larger examples through modularity and reuse.

4.4 The string interface

The string interface defines an entire Turing program by parsing a collection of strings. Its syntax is intentionally designed to resemble the notation commonly found in textbooks.

1. The tape symbols must be either characters or non-negative integers.
2. The predefined halting states are represented by the characters Z, T, F, and X, corresponding to HALT, TRUE, FALSE, and FAIL, respectively.
3. The head movements are represented by the characters L, R, and S, corresponding to LEFT, RIGHT, and STAY, respectively.
4. The special symbols *, #, and = represent ALL (every symbol, including the blank), every symbol in the alphabet (excluding the blank), and SAME (leave the symbol unchanged), respectively.
5. A set of states or symbols may be matched using a brace-enclosed, comma-separated list with no embedded spaces. For example, if the alphabet is `Alphabet Decimal( 10 )`, the even digits are matched by `{0,2,4,6,8}`.

A Turing program may optionally be given an identifier as the first argument to its constructor. Named programs may subsequently be called from within other Turing programs.

An ordinary transition is specified by the current state `S`, the symbol `c` currently under the head, the next state `T`, the symbol `d` to be written, and the head movement:

```
" S c -> T d L "
```

A transition may also invoke another named Turing program:

```
" S c -> call Program_name T->5 F->6 "
```

In this example, the program `Program_name` is executed beginning at its initial state. If it terminates in state `TRUE`, execution of the calling program continues in State 5; if it terminates in state `FALSE`, execution continues in State 6. Any user-defined halting states of the called program must be mapped explicitly, while predefined halting states may either be mapped explicitly or be shared halting states of both programs.

These transition descriptions are passed to the constructor as an initializer list. The additional whitespace shown in the examples is optional and is included only to improve readability.

For example, the Turing program that erases all symbols to the right of the head may be defined as:

```
Alphabet A{ "01" };
Turing_program CLEAN_TO_RIGHT{ "Clean_to_right", A, {
    " 0 _ -> Z _ S ", // If we are on a blank, halt.
    " 0 # -> 0 _ R "  // Otherwise, erase the symbol and move right.
} };
```

The following program determines whether the head is positioned over the only non-blank symbol on the tape:

```
Alphabet Binary{ "01" };
Turing_program IS_ONE_BIT{ "Is_one_bit", Binary, {
    " 0 _ -> F _ S ",
    " 0 # -> 1 = R ",
    " 1 _ -> 2 _ L ",
    " 1 # -> F = L ",
    " 2 # -> 3 = L ",
    " 3 _ -> T _ R ",
    " 3 # -> F = R "
}, { TRUE, FALSE } };
```

Multiple states may be matched using a brace-enclosed, comma-separated list such as $\{0,1\}$. Likewise, multiple symbols may be matched using a list such as $\{0,1,*\}$, where $*$ represents the BLANK symbol. The wildcard $*$ matches every symbol, including the blank, while $\#$ matches every symbol in the alphabet but not the blank. Finally, $=$ indicates that the current symbol is to remain unchanged.

5 Example Turing programs

The following are examples of Turing programs that perform various operations. We only use the string interface, as it is easier to visualize the transitions.

5.1 Move to the left- or right-most symbol

This program moves to the left- or right-most symbol of the word that is either under the head, or immediate next to a BLANK in the same direction.

```
Turing_program Move_to_left_most_symbol{ "move_to_left_most_symbol", Alphabet, {
    " 0 * -> 1 = L ",    // Move one character to the left
    " 1 # -> 1 = L ",    // So long as you are over a symbol, move to the left
    " 1 _ -> Z _ R "     // Having reached a blank, move right and halt
} };

Turing_program Move_to_right_most_symbol{ "move_to_right_most_symbol", Alphabet, {
    " 0 * -> 1 = R ",    // Move one character to the left
    " 1 # -> 1 = R ",    // So long as you are over a symbol, move to the right
    " 1 _ -> Z _ L "     // Having reached a blank, move left and halt
} };
```

We do not move until we find a word as this may place the program in an infinite loop, as may happen with the program below if there are no symbols to the left.

```
Turing_program Alternative_move_to_left_most_symbol{ "alt_move_to_left_most_symbol", Alphabet, {
    " 0 _ -> 0 _ L ", // Keep moving to the left until you reach a symbol
    " 0 # -> 1 = L ", // Having reached a symbol, move to the left and -> State 1
    " 1 # -> 1 = L ", // So long as you are over a symbol, move to the left
    " 1 _ -> Z _ R " // Having reached a blank, move right and halt
} };
```

5.2 Is a word a non-negative binary number?

This returns TRUE if the head is currently any bit of a binary number, including the special case of '0', where the most significant bit is a '1'. If true, the head will be in the location of the least significant bit.

```
Turing_program Is_non_negative_binary{ "Is_non_negative_binary", Alphabet, {
    "0 _ -> F _ S",
    "0 {0,1} -> call move_to_right_most_symbol Z->1",
    "1 0 -> 2 0 L",
    "1 1 -> 4 1 L",
    "2 _ -> T _ R",
    "2 0 -> 3 0 L",
    "2 1 -> 4 1 L",
    "3 _ -> 5 _ R",
    "3 0 -> 3 0 L",
    "3 1 -> 4 1 L",
    "4 _ -> 6 _ R",
}
```



```

    "4 0 -> 3 0 L",
    "4 1 -> 4 1 L",
    "5 {0,1} -> call move_to_right_most_symbol Z->F",
    "6 {0,1} -> call move_to_right_most_symbol Z->T"
}, {TRUE, FALSE} };

```

5.3 Is a word a byte?

This returns TRUE if the head is currently over a byte and moves the head to the least-significant bit.

```

Turing_program Is_byte{ "Is_byte", Alphabet, {
    "0 _ -> F _ S",
    "0 {0,1} -> call move_to_left_most_symbol Z->1",
    "1 {0,1} -> 2 = R",
    "1 _ -> F = L",
    "2 {0,1} -> 3 = R",
    "2 _ -> F = L",
    "3 {0,1} -> 4 = R",
    "3 _ -> F = L",
    "4 {0,1} -> 5 = R",
    "4 _ -> F = L",
    "5 {0,1} -> 6 = R",
    "5 _ -> F = L",
    "6 {0,1} -> 7 = R",
    "6 _ -> F = L",
    "7 {0,1} -> 8 = R",
    "7 _ -> F = L",
    "8 {0,1} -> 9 = R",
    "8 _ -> F = L",
    "9 _ -> T = L",
    "9 {0,1} -> call move_to_right_most_symbol Z->F"
}, {TRUE, FALSE} };

```

5.4 The complement and two's complement of a binary number

This program assumes that the head is on a binary number and you want to complement it. The head will end at the least-significant bit.

```

Turing_program Complement{ "Complement", Alphabet, {
    "0 {0,1} -> call move_to_left_most_symbol Z->1",
    "1 0 -> 1 1 R",
    "1 1 -> 1 0 R",
    "1 _ -> Z _ L"
} };

Turing_program Twos_complement{ "Twos_complement", Alphabet, {
    "0 {0,1} -> call move_to_right_most_symbol Z->1",
    "1 0 -> 1 0 L",
    "1 1 -> 2 1 L",

```

```
    "1 _ -> call move_to_right_most_symbol",  
    "2 0 -> 2 1 L",  
    "2 1 -> 2 0 L",  
    "2 _ -> call move_to_right_most_symbol",  
} };
```

Here is

```
Tape t1{ Binary, "01011100", 3 }, t2{ Binary, "11011100", 4 }, t3{ Binary, "11011101", 5 };
std::cout << t1 << " " << t2 << " " << t3 << std::endl;

std::cout << "All of these are bytes:" << std::endl;
std::cout << Is_byte( t1 ) << std::endl;
std::cout << Is_byte( t2 ) << std::endl;
std::cout << Is_byte( t3 ) << std::endl << std::endl;

std::cout << "The first does not have a leading '1' and is not zero:" << std::endl;
std::cout << Is_non_negative_binary( t1 ) << std::endl;
std::cout << Is_non_negative_binary( t2 ) << std::endl;
std::cout << Is_non_negative_binary( t3 ) << std::endl << std::endl;

std::cout << "The complement and 2's complement of these bytes:" << std::endl;
std::cout << t1 << std::endl;
Complement( t1 );
std::cout << t1 << std::endl;
Complement( t1 );
Twos_complement( t1 );
std::cout << t1 << std::endl << std::endl;

std::cout << t2 << std::endl;
Complement( t2 );
std::cout << t2 << std::endl;
Complement( t2 );
Twos_complement( t2 );
std::cout << t2 << std::endl << std::endl;

std::cout << t3 << std::endl;
Complement( t3 );
std::cout << t3 << std::endl;
Complement( t3 );
Twos_complement( t3 );
std::cout << t3 << std::endl << std::endl;

Tape t4{ Binary, "0" }, t5{ Binary, "1" }, t6{ Binary, "10" },
      t7{ Binary, "01" }, t8{ Binary, "11" };
std::cout << t4 << " " << t5 << " " << t6 << " " << t7 << " " << t8 << std::endl;

std::cout << "None of these are bytes:" << std::endl;
std::cout << Is_byte( t4 ) << std::endl;
std::cout << Is_byte( t5 ) << std::endl;
std::cout << Is_byte( t6 ) << std::endl;
std::cout << Is_byte( t7 ) << std::endl;
std::cout << Is_byte( t8 ) << std::endl;

std::cout << "The fourth does not have a leading '1' and is not zero:" << std::endl;
std::cout << Is_non_negative_binary( t4 ) << std::endl;
std::cout << Is_non_negative_binary( t5 ) << std::endl;
std::cout << Is_non_negative_binary( t6 ) << std::endl;
std::cout << Is_non_negative_binary( t7 ) << std::endl;
std::cout << Is_non_negative_binary( t8 ) << std::endl;
```